

Flexing RISC-V Instruction Subset Processors to Extreme Edge

Alireza Raisiardi
Pragmatic Semiconductor
Cambridge, United Kingdom
KU Leuven
Leuven, Belgium
araisiardi@pragmaticsemi.com

Konstantinos Iordanou
Pragmatic Semiconductor
Cambridge, United Kingdom
kiordanou@pragmaticsemi.com

Jedrzej Kufel
Pragmatic Semiconductor
Cambridge, United Kingdom
jkufel@pragmaticsemi.com

Kowshik Gudimetla
Pragmatic Semiconductor
Cambridge, United Kingdom
kgowshik@pragmaticsemi.com

Kris Myny
KU Leuven
Leuven, Belgium
kris.myny@kuleuven.be

Emre Ozer
Pragmatic Semiconductor
Cambridge, United Kingdom
eozer@pragmaticsemi.com

Abstract

This paper presents an automated approach for designing processors that support a subset of the RISC-V instruction set architecture (ISA) for a new class of applications at Extreme Edge. The electronics used in extreme edge applications must be area and power-efficient, but also provide additional qualities, such as low cost, conformability, comfort and sustainability. Flexible electronics, rather than silicon-based electronics, will be able to meet the above qualities. For this purpose, we propose a methodology for generating RISC-V instruction subset processors (RISSPs) tailored to these applications and implementing them as flexible integrated circuits (FlexICs). The methodology makes verification an integral part of the processor design by treating each instruction in the ISA as a discrete, fully functional, pre-verified hardware block. It automatically builds a custom processor by stitching together the instruction hardware blocks required by an application or a set of applications in a specific domain. We generate RISSPs using the proposed methodology for three extreme edge applications, and embedded applications from the *Embench* benchmark suite. When synthesized, RISSPs can achieve 8-to-43% reduction in area and 3-to-30% reduction in power compared to a processor supporting the full RISC-V ISA, and are also on average ~40 times more energy efficient than *Serv* - the world's smallest 32-bit RISC-V processor. When physically implemented as FlexICs, the three extreme edge RISSPs achieve up to 42% area and 21% power savings with respect to the full RISC-V processor.

Keywords

RISC-V, Domain-specific Processors, Extreme Edge, Printed and Flexible Electronics and FlexIC

ACM Reference Format:

Alireza Raisiardi, Konstantinos Iordanou, Jedrzej Kufel, Kowshik Gudimetla, Kris Myny, and Emre Ozer. 2025. Flexing RISC-V Instruction Subset Processors to Extreme Edge. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*, October 18–22, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3725843.3756036>

1 Introduction

A new class of emerging applications requires embedded electronics capable of sensing, computing, and communicating while also offering qualitative features such as low cost, conformability, comfort, biocompatibility, and sustainability in addition to low power consumption. This new class of applications is now described as "Extreme Edge" which extends beyond edge computing today [12].

Extreme edge applications can be classified into two main categories: 1) **Short-lived applications** that have short lifetimes. Item-level intelligence, Fast Moving Consumer Goods (FMCG) and single-use healthcare products are examples of short-lived applications. Since electronics are part of the product itself, they will have a short lifetime, and any firmware/software update to the electronics will not be very likely. For instance, a smart dressing product has built-in electronics to monitor wound healing conditions. The lifetime of the dressing is in the range of days before it is replaced. The dressing and its built-in electronics will be disposed. 2) **Long-lasting applications** that have much longer lifespan. Smart textile, wearable healthcare and environmental/agriculture monitoring are examples of long-lasting extreme edge applications. For instance, the electronics embedded into a smart garment can be monitoring the health status and performance of a person. The garment will be rewashed and reused in a lifespan of several months before it wears out, and its built-in electronics may receive software/firmware updates during its lifetime. **Table 1** summarizes the application domains and features of these two categories.

Conventional silicon-based electronics are unable to meet the requirements of extreme edge applications because they do not satisfy many of the required features. For example, smart labels in asset tracking and smart packages in FMCG are ultra cost-sensitive applications where electronics embedded into them must not cost more than a few cents per unit to ensure economic viability. An electrocardiogram (ECG) patch requires a comfortable and conformable form

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MICRO '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1573-0/25/10

<https://doi.org/10.1145/3725843.3756036>

Table 1: Classification of extreme edge applications, application domains and distinct characteristics

Category	Short-lived	Long-lasting
Domains	Item-level intelligence (e.g., smart labels), FMCG (e.g., smart food/drink packaging), Single-use healthcare (e.g., patches, wound dressings)	Smart textile (e.g., smart garment and rugs), Wearable healthcare (e.g., implantables and ingestibles), Environmental/agriculture monitoring (e.g., smart plant patches)
Required Qualitative Features	Ultra Low-cost, Conformability, Patient Comfort and Sustainability	Low-cost, Conformability, Biocompatibility and Sustainability
Application Lifespan	Days/Weeks	Months/Years
Requirement of Software and Firmware Updates	Not likely	Very likely

factor in electronics for the patient where conventional bulky electronics will cause discomfort. An implantable or ingestible device must contain electronics that are biocompatible and conformable. The volume of extreme edge products in terms of units is enormous, measured in a few trillions annually [19, 21, 35, 36]. If electronic devices are embedded into such volume of products, they must be manufactured in sustainable semiconductor fabs that have a low environmental footprint.

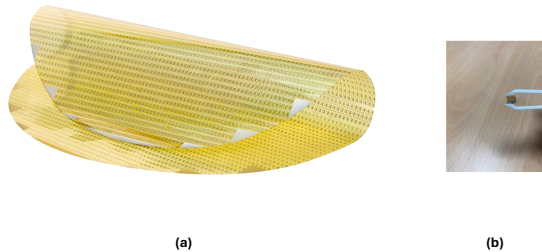


Figure 1: FlexIC technology. (a) A 200 mm wafer on a polyimide substrate, (b) A FlexIC is an integrated circuit manufactured on a polyimide wafer substrate. The semiconductor material is metal-oxide called Indium-Gallium-Zinc-Oxide (IGZO) that can be manufactured at low temperatures. The image of the FlexIC diced from the polyimide wafer is shown. It is an ultra-thin (30 μm) and physically flexible or bendable.

Flexible electronics [26] is an alternative technology that meets many of the qualities required by extreme edge. The aim is to produce electronic components that include sensors, compute engines, communication interfaces, batteries using low-cost printing, and lithography techniques on flexible substrates (e.g., plastic or paper). For example, flexible integrated circuits (FlexICs) [34] can be

fabricated using metal-oxide thin-film transistor technology on a polyimide wafer, as shown in **Figure 1**.

The chips are ultra-thin, conformable, bendable and ultra-low cost [31], and the semiconductor fabs that manufacture FlexICs have significantly lower carbon footprint per FlexIC than an equivalent Si chip [1]. Because the fabrication process is low-cost and fast, the development of custom FlexICs specifically designed for an application or domain can be faster and lower in cost than Silicon technology.

The performance requirements of the two extreme edge categories are not demanding where a sensor or an array of sensors are connected to an analogue frontend to pre-process sensor raw signals that are converted into digital data and handled by a digital backend to translate them into knowledge. The knowledge is stored and/or directly transmitted to the external world through a communication interface (e.g., RF). Data sampling rates required by sensors are 200 Hz[5], which implies that digital backend processing speeds can be in the range of Hz for many extreme edge applications or kHz for applications that require faster response.

Natively flexible smart systems [11] consisting of flexible electronic components, such as sensors, displays and batteries coupled with compute engines, memory and communication units, are a natural fit for extreme edge. Both general-purpose processors and domain-specific processing engines have been developed as FlexICs. For instance, the studies in [3, 4, 7, 24, 25, 31] demonstrated natively flexible 4-bit, 8-bit and 32-bit general-purpose processors whilst others proposed natively flexible domain-specific processing engines in the domains of RFID communication [23, 42], machine learning [20, 27–30, 33], security [5], and healthcare [32, 43]. These natively flexible processors have been shown to operate at the kHz range reaching up to 100 kHz, which is sufficient to meet the performance requirements of extreme edge applications.

Although designing a domain-specific processing engine is more efficient than a general-purpose processor, it takes a considerable time to design, test and verify the design if the ASIC approach is taken for domain-specific processing. An alternative is to use an application-specific instruction processor (ASIP) [2, 40], where the instruction sets are customized based on the application or domain. However, this approach needs to generate not only the ASIP but also the associated software toolset, such as a compiler, linker and simulator.

In this paper, we propose a methodology for automatically generating domain-specific processors for extreme edge applications. The proposed methodology relies on the open standard RISC-V [41] instruction set architecture (ISA) and generates processors that use only a subset of the RISC-V full ISA (e.g., RV32I). We observe (see *Section 4*) that extreme edge and extreme edge-like embedded applications only use 24–86% of the full RISC-V ISA. Thus, a processor supporting a subset of the full RISC-V ISA can be more efficient in terms of performance, power and area (PPA) because the logic related to the unused instructions is avoided. We define them as **RISC-V Instruction SubSet Processors** or **RISSPs**.

The RISSP generation methodology takes a very different approach to a processor microarchitecture where verification is an integral part of the design process. The main concept is to design a RISSP from pre-verified building blocks rather than the traditional design-followed-by-verification approach. First, the semantics of

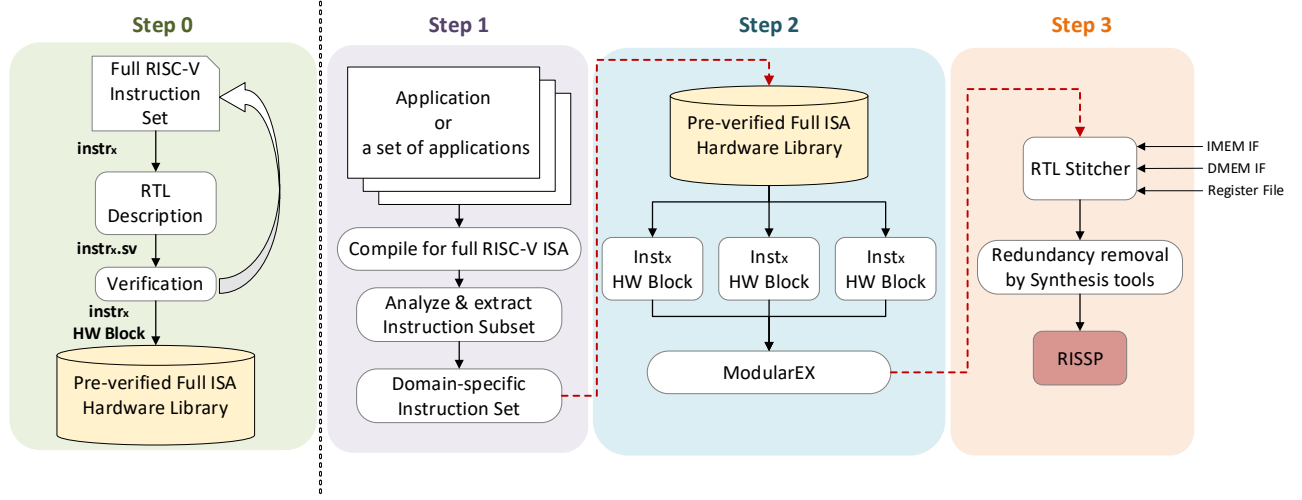


Figure 2: RISSP generation methodology. STEP 0: Development of the pre-verified full ISA hardware library, which is the one-time development effort of the methodology; STEP 1: Generation of domain-specific instruction subset from an application or a set of applications from a domain; STEP 2: Pulling instruction hardware blocks for the domain-specific instruction set from the pre-verified full ISA hardware library, and formation of ModularEX; STEP 3: Construction of the RISSP by stitching ModularEX with fixed units like memory interfaces and the register file

all instructions in the ISA are written in RTL as **instruction hardware blocks**. Then, they are formally verified, and stored into a library. A RISSP is formed by extracting the required instruction hardware blocks from the library, composing them in a modular manner and reducing the verification effort significantly. Since extreme edge applications do not require high-speed operating clock frequencies, single-cycle non-pipelined RISSPs suffice to meet their performance requirements.

The RISSP generation methodology avoids the significant effort of generating proprietary software toolsets specific to ASIPs by relying on the RISC-V software ecosystem. For short-lived extreme edge applications, we predict that RISSPs do not need software updates due to the short life of electronics embedded in the extreme edge device. For long-lasting extreme edge applications, occasional software updates to RISSPs are needed. In this context, the updated application(s) need to be recompiled but the generated code must be constrained to the instruction subset supported by the target RISSP. This requires several modifications in the RISC-V compiler (e.g., gcc) backend to constrain code generation to the instruction subset of the RISSP. However, these modifications involve various challenges at multiple levels of the compiler stack. Instead, we develop a Generative AI-based tool that receives the assembly code of the updated application(s) recompiled to the full ISA, and rewrites the new instructions that are not supported by the RISSP in terms of the instruction subset.

The proposed methodology can automatically generate RISSPs that are 8-to-43% smaller and 3-to-30% lower power, when synthesized, compared to a processor supporting the full RISC-V ISA, also generated by the same methodology, and on average about 40 times more energy efficient than *Serv* [38]. When physically implemented as FlexICs, the extreme edge RISSPs achieve up to 42% area and 21% power savings relative to the full RISC-V processor.

The contributions of this paper are summarized as follows:

- (1) Automatic generation of RISC-V instruction subset processors (RISSPs) that implement only the subset of the RISC-V ISA targeting extreme edge applications, whilst still relying on the existing software ecosystem
- (2) Design of a RISSP using a pre-verified instruction hardware block library to significantly reduce the verification time and time-to-market of RISSPs
- (3) A Generative AI based tool to retarget the new instructions in the updated application(s) to the instructions in the instruction subset supported by RISSP for long-lasting extreme edge applications

The remainder of this paper is structured as follows: *Section 2* discusses the related work in processor customization. *Section 3* presents the proposed methodology of generating RISSPs. *Section 4* describes the experimental evaluation and also presents the physical implementation results of RISSPs as FlexICs. *Section 5* introduces the Generative AI-based code recompilation/retarget framework. Finally, *Section 6* concludes the paper.

2 Related Work

There are two main approaches in the design of domain-specific processors: i) Top-down or subtractive customization approach where the customization is performed over an already designed processor by removing the gates that are not used by the target application; ii) Bottom-up or additive approach where the customization is performed at the design level by placing in the hardware blocks required by the applications.

PDAT [6] and Bespoke processors [10] are two examples of a subtractive customization approach. PDAT converts a baseline processor into a standard netlist and uses property assertions to

determine whether a specific gate is utilized during the execution of an embedded application. It exploits gate-level property checking to replace unused cells with wires if the relevant property is not asserted. Bespoke processors focus on the development of processing hardware from a baseline processor customized to a specific application. It uses input-independent gate activity analysis to remove the gates from the baseline processor that do not toggle during execution, resulting in a bespoke processor. The main disadvantage of the subtractive approach is that the opportunity of finding and removing a large number of unused gates and hardware blocks in a baseline processor is limited because processors are typically designed as lean as possible. For example, an arithmetic logic unit (ALU) is shared among many instructions, and these subtractive approaches are conservative to remove the portions of the ALU even if some of the ALU-using instructions can be removed.

ASIPs [2, 18, 22, 40] use an additive customization approach in which they create the custom instruction set for the target application or domain and then generate the ASIP that executes the custom instruction set. A complete software toolchain (e.g., a compiler, assembler, linker and simulator) is also generated to support the ASIP, which is an overhead. They adopt conventional processor microarchitecture techniques, and verification is not an integral part of the design. Conventional verification strategies must be used after generating ASIPs.

On the other hand, verification is an integral part of the RISSP generation methodology, and RISSPs are created in a bottom-up approach using the novel pre-verified instruction hardware block library concept reducing the verification effort of the generated custom processor. In addition, RISSPs rely on open-source software toolchain of the RISC-V software ecosystem rather than producing a new software toolchain for a proprietary ISA like in ASIPs.

3 Methodology of RISSP Generation

RISC-V [41] is one of the most popular ISAs that is rapidly established in various embedded domains. A key design principle of the RISC-V ISA is its modularity allowing computer architects to design custom processors based on the requirements of the target application. Additionally, the built-in support for custom instructions offers a significant advantage compared to other ISAs. While a modular ISA allows independent instruction groups to be separated and combined based on the application requirements, the modularity of the ISA implementation depends on how these instruction groups or extensions are designed in hardware. It is important to note that a modular ISA does not necessarily infer that the underlying implementation of the ISA is modular. Unless the ISA implementation inherently supports “instruction-level modularity”, the only way to remove a subset of the ISA is to modify the actual RTL design. However, removing support for a subset of the ISA from the actual ISA implementation is not straightforward and requires a deep understanding of the RTL implementation of the processor supporting the ISA. This process may introduce errors in the design and extensive verification is necessary to ensure that the modifications maintain the functionality.

One of the key benefits of this work is to enhance the instruction-level modularity of the ISA implementation by defining instructions as independent, fully functional, pre-verified RTL blocks.

Table 2: A high-level overview of instruction hardware blocks of the RISC-V RV32I/E ISA

Type	Instructions	Block Diagram
B-type	beq, bne, blt, bge, bltu, bgeu	
R-type	add, sub, sll, slt, sltu, xor, srl, sra, or, and	
I-type	lb, lh, lw, lbu, lhu, addi, slli, slti, sltiu, xori, srli, srai, ori, andi, jalr	
S-type	sb, sh, sw	
U-type	lui, auipc	
J-type	jal	

3.1 Concept of Instruction Hardware Blocks

The RISSP generation methodology relies on the RISC-V ISA and its software toolchain. A RISSP is automatically generated for a given application or a domain of similar applications, as shown in **Figure 2**. It starts with the development of the pre-verified full ISA hardware library (see Step 0). This is a one-time effort for a given ISA. The pre-verified full ISA hardware library contains “formally verified” instruction hardware blocks, and is, in concept, similar to the standard cell library designed for a process technology used in chip design and implementation, which will be described in *Section 3.4.1*. Next, we characterize the application or the domain of applications by compiling them to the target RISC-V ISA (in our case RV32E) to identify the domain-specific instruction subset (see Step 1). The hardware description of each instruction in the instruction subset is called “instruction hardware block” written in RTL (i.e., SystemVerilog), and extracted from a pre-verified full ISA hardware library of all instructions in RISC-V ISA.

Each instruction in the RISC-V ISA can be thought of as a discrete, fully functional hardware block. **Table 2** highlights the block diagram of each instruction type in the ISA. The semantics of each

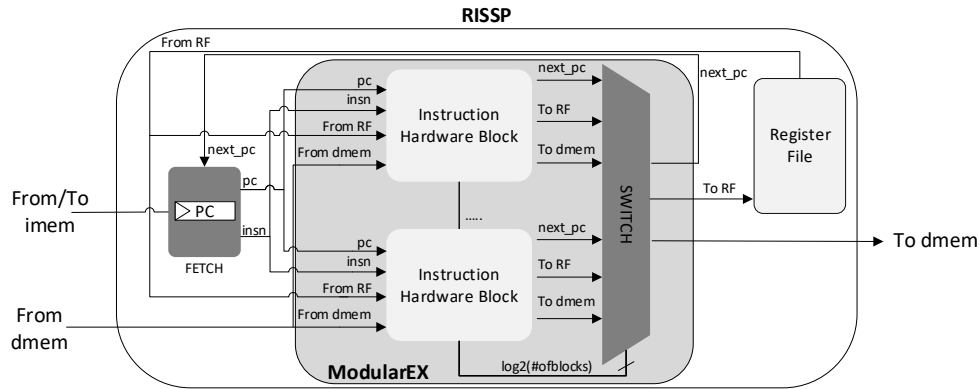


Figure 3: A single-cycle RISSP microarchitecture created by stitching together ModularEX, the fixed units (i.e., Fetch and RF) and Memory Interfaces

instruction is converted into its hardware description in RTL, along with a glue logic interfacing with the Register File (RF) and the memory. The extracted instruction hardware blocks are then automatically combined to build the Modular Execution Unit (ModularEX).

3.2 Modular Execution Unit

ModularEX is the main execution unit of the generated RISSP. It consists of a switch that handles the instruction hardware blocks and their interfaces. The switch is automatically generated and ensures that the outputs of the instruction-under-execution module are triggered. It is a simple case statement in SystemVerilog with N cases, where N is the number of the instructions in the sub-ISA. The instruction hardware blocks are integrated with the switch into ModularEX (fully combinational design).

In Step 3 - **Figure 2**, ModularEX is stitched to the memory interfaces and the RF to generate an unoptimised RISSP, which is an input to the hardware synthesis tool that optimizes it by maximizing sharing inside ModularEX. The output is the final RISSP tailored to the application or the domain under consideration, which is fully programmable within the boundaries of its instruction subset.

3.3 RISSP Construction

Figure 3 shows the construction of a RISSP by stitching ModularEX with the rest of the fixed units such as the fetch unit and the RF as well as memory interfaces. By construction, a RISSP has a single-cycle processor microarchitecture where an instruction is fetched from the instruction memory using a 32-bit Program Counter (PC), and sent to the ModularEX unit. Then, the switch selects the actual instruction hardware block of the fetched instruction executing it at the same cycle. In other words, the switch performs as a partial decoder that selects which block will be enabled at every clock cycle. The full decoding of the instruction is performed inside each instruction block.

The key benefit of the RISSP generation methodology is that it does not require any expertise in microarchitecture, RTL design or gate netlists. The only input of the framework is the domain-specific instruction subset identified by the application or the domain. All design optimization decisions are taken by the synthesis tool rather

than optimizing the design manually. This is a conscious decision because contemporary commercial/non-commercial hardware synthesis tools are very efficient in discovering redundancies and eliminating them. For instance, the synthesis tool will optimize the gate netlists by maximizing the resource sharing if multiple instruction hardware blocks have common operations among them (e.g., design flattening). Any logic optimization performed during synthesis will be formally verified using equivalence checking by the synthesis tool. This approach may not lead to "minimal area/power" but minimizes the design and verification time.

3.4 Verification

Verification is a critical and complex phase in processor design. The fundamental benefit of the proposed methodology is the ability to streamline and simplify the verification process.

3.4.1 Development of the Pre-verified Full ISA Hardware Library. Generating a RISSP using the novel concept of a pre-verified ISA hardware library significantly reduces the verification complexity of RISSP. The full ISA hardware library has been extensively verified in multiple ways including formal verification. The development of the pre-verified ISA hardware library is a one-time verification effort that becomes the non-recurring-engineering (NRE) cost of RISSP. The NRE is amortized by the number of RISSPs shipped during the lifetime of this library. In contrast, any new design in conventional CPU/ASIP methodologies requires design and verification, i.e., verification effort becomes a recurring cost.

The full ISA hardware library is a collection of instruction hardware blocks written in SystemVerilog. The library supports the RISC-V RV32I/E instruction set and is fully extendable to support other groups of RISC-V instructions or even custom instructions. Each instruction hardware block is formally verified against its specification in the ISA. This is a significant step towards instruction modularity in the RISSP design and its simplified verification. One of the key advantages of this approach is that verification of instruction hardware blocks is integrated into the RISSP development process. Every instruction hardware block is formally pre-verified before it is used for the generation of a RISSP.

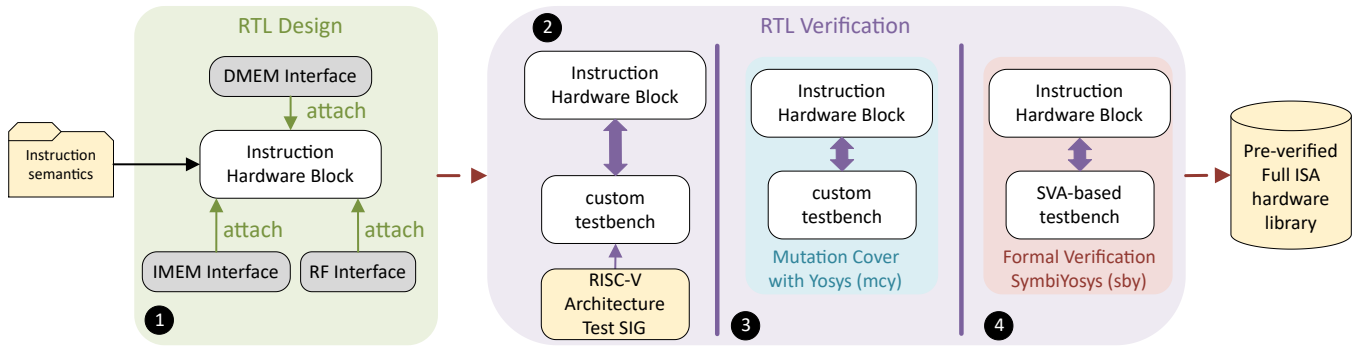


Figure 4: The development of the pre-verified full ISA hardware library

Figure 4 presents the development process of the pre-verified full ISA RTL library. Starting with the RTL design, the instruction semantics are combined with pre-developed standard interfaces - used for communication with the rest of the processor - to define the actual instruction hardware block (Figure 4 - ①). Following the design phase, there are three distinctive verification steps before including the new instruction hardware block in the library. This process includes: (a) the development of a testbench that automatically verifies the instruction hardware block using standard test cases (Figure 4 - ②), (b) self-checking of the testbench to ensure that can verify the instruction hardware block under test (Figure 4 - ③), and (c) the formal verification of the instruction hardware block using SystemVerilog Assertions (SVA) (Figure 4 - ④).

For functional verification, individual test cases from the RISC-V Foundation Architecture Test SIG [15] are automatically extracted for each instruction hardware block, and custom testbenches have been developed to verify these test cases. Specifically, RISC-V Foundation Architecture Test Suite provides an extensive set of test cases for each instruction, developed in Assembly, ensures that the RISC-V specification has been interpreted correctly and the implementation of the target processor is RISC-V Architecture test compliant. The toolflow automatically isolates the tests cases for each instruction, transforms them in a binary format, and passes the test vector to the instruction hardware block through our custom testbenches. This process iteratively verifies each instruction hardware block for the complete set of test cases.

To further extend our verification framework of the library, each testbench is evaluated with MCY tool from YosysHQ suite [17]. MCY is one of the most popular tools for verification primarily because it offers the ability to generate several mutations of the target RTL design (in our case the implemented instruction blocks). The tool automatically selects only the mutated designs that can cause an important change in the output of the design by filtering them using formal verification techniques. The final step is to automatically check that the testbench can detect and fail for the mutations that can cause an important change in the output of the target design. Under this environment, we ensure that our testbenches are capable of verifying the target instruction hardware blocks.

In terms of formal verification, the full ISA library is formally verified using SymbiYosys [14] (sby), a frontend for Yosys for Formal Verification. For every instruction hardware block, a set of SVA are

developed to ensure that the instruction semantics are satisfied. Also, additional assertions are used to validate the behavior of the interfaces. This approach simplifies the verification process of a RISSP, as each instruction hardware block is already independently and formally verified. Moreover, the process benefits from the well-defined semantics of each instruction in the ISA.

Instruction hardware block representations of all instructions in the ISA are functionally and formally verified and are stored in the pre-verified full ISA hardware library. When constructing a RISSP, instruction hardware blocks from the library will be stitched together to build the ModularEX that does not require any further verification. Therefore, the pre-verified full ISA hardware library concept is similar to the concept of standard cell libraries in VLSI design where the standard cell library for a foundry process is created once and re-used in many chip designs. Each cell is a fully verified building block in a finer granularity.

3.4.2 Integration-level Verification. The pre-verified ModularEX is integrated into the pre-verified fixed units, such as fetch and register file to construct the RISSP. Various verification processes and unit-tests have been developed to ensure the correctness of the integrated RISSP. They include custom domain checks at various points, such as checks at instruction and register level to verify that each instruction is fetched and executed correctly. Additionally, RISC-V compatibility is tested with *RISCOF*, an architectural test framework [16]. *RISCOF* automatically chooses a set of test cases for the RV32E ISA, the RISSP executes these test cases, and writes a signature to a file. Then, this signature is compared with a signature of a reference model that has been obtained from a RISC-V simulator, such as *Spike* [37].

In terms of formal verification at processor-level, RISSP has been verified up to a specific depth with *riscv-formal* [13]. This framework is an open-source standard for the formal verification of RISC-V processors and is based on the RISC-V Formal Interface (RVFI), a standard interface for processors to communicate and verification tools. Specifically, RISSP implements the RVFI interface that mainly tracks valid instructions, register file data and addresses, PC changes and memory accesses. With *riscv-formal*, we perform various and extensive checks at instruction level for correct execution, checks to ensure that the registers maintain consistency, and PC checks that verify the correct PC values before and after the execution of an instruction.

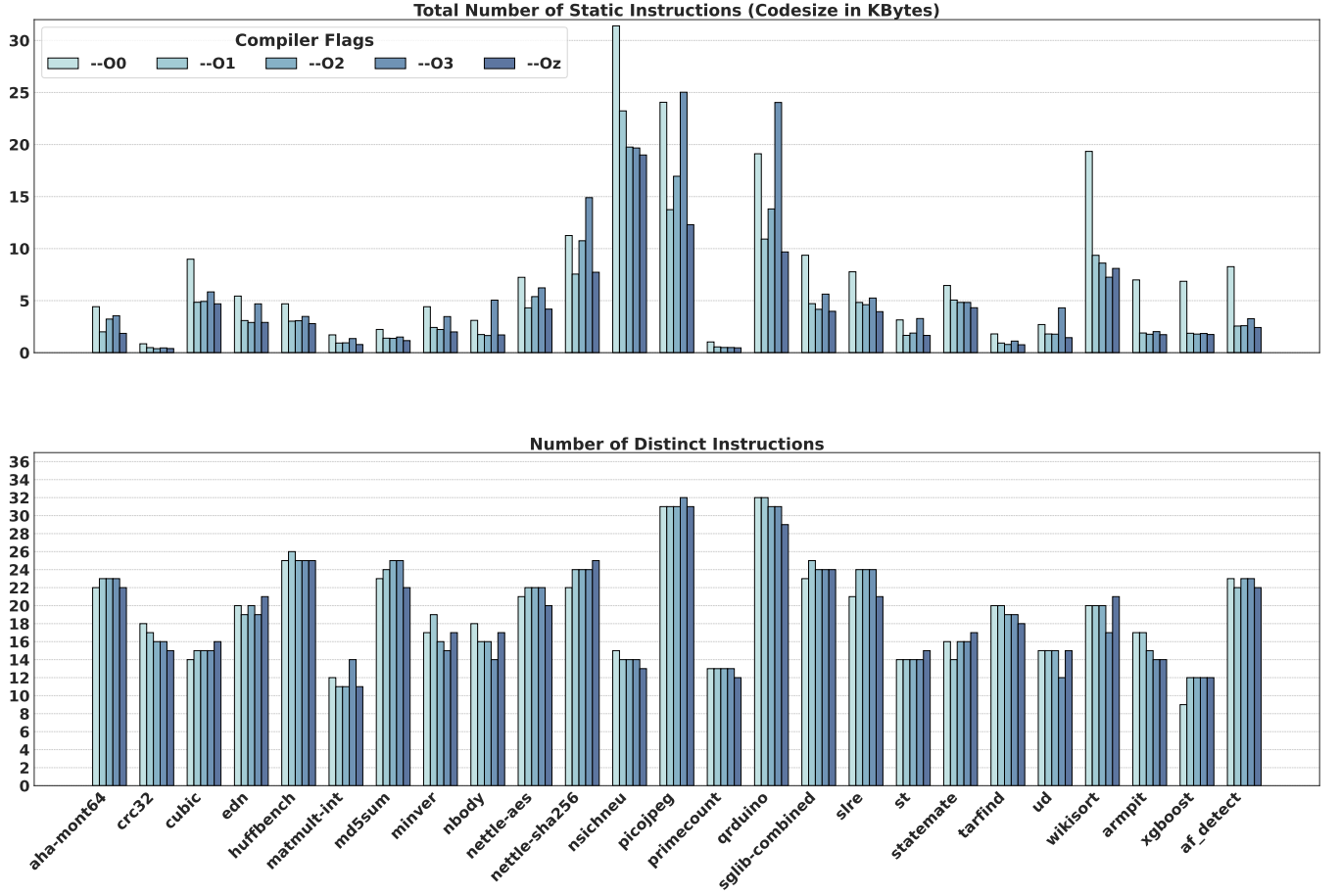


Figure 5: Results of instruction profiling and characterization of the three extreme edge applications, and the benchmarks in *Embench* using different compiler optimization flags. The top diagram presents the codesize in KBytes and the bottom diagram illustrates the number of distinct instructions from the RV32E ISA across different optimization flags.

4 Experimental Evaluation

One of the key challenges in evaluating the RISSP generation methodology is the lack of a representative benchmark suite for extreme edge. We select two extreme edge applications previously demonstrated in the FlexIC technology [29] [32], and also an ML kernel *XGBoost* (eXtreme Gradient Boosting) [9] as one of the key ML algorithms used in many classification tasks as an extreme edge kernel.

Extreme edge application 1: Armpit malodour classification or *armpit* [29] is a real-world application in the domain of deodorant industry that advocates the use of flexible organic sensors to detect volatile organic components in odour with a sensor readout interface and machine learning hardware implemented as a custom FlexIC to score malodour in a human armpit. It is a representative example of a natively flexible smart system [11] consisting of all flexible components from sensors to processing. Two decision trees (one for each gender) are implemented in hardware to classify armpit malodour. The decision trees in the armpit application are

used as the benchmark in this paper. The C code of the decision trees was compiled to RV32E to generate a RISSP for the application.

Extreme edge application 2: Atrial fibrillation (AF) detection or *af_detect* [32] is another real-world application in the domain of ECG that uses flexible electronics to detect AF events in a wearable ECG patch. An AF prediction algorithm called “Approximate Pair Presence Tracking” or APPT was implemented as a custom FlexIC. APPT has three stages: i) detection of R peaks in the ECG signal; ii) calculation of RR intervals (the interval between two R peaks) and delta of RR intervals; iii) a Bloom filter-based binary predictor (AF versus non-AF) using a RR and delta RR interval map. The APPT algorithm in *af_detect* is used as the benchmark in this paper, which is compiled for RV32E to generate a RISSP.

Extreme edge kernel: *XGBoost* [9] is known for its performance in classification and regression problems. It is a more accurate and scalable implementation of gradient boosting. For datasets and problems with simpler requirements, decision trees can be mapped on edge devices in the form of C/C++ or on custom hardware implementation processing units to perform ML classification problems, e.g., *armpit*. We use *XGBoost* to extract a decision tree from a dataset

Table 3: List of distinct instructions per application when compiled with -O2 option

Application	List of Distinct Instructions
aha-mont64	[add, addi, and, andi, beq, bge, bgeu, bltu, bne, jal, jalr, lui, lw, or, slli, sltiu, sltu, srai, srli, sub, sw, xor, xori]
crc32	[add, addi, andi, bge, bne, jal, jalr, lui, lw, slli, sltiu, srli, sub, sw, xor, xori]
cubic	[addi, and, andi, beq, bge, blt, bne, jal, jalr, lui, lw, slti, sltiu, sw, xor]
edn	[add, addi, andi, beq, bge, bne, jal, jalr, lh, lhu, lui, lw, sh, slli, sltiu, sra, srai, srli, sub, sw]
huffbench	[add, addi, and, andi, beq, bge, bgeu, blt, bltu, bne, jal, jalr, lbu, lui, lw, or, ori, sb, sll, slli, sltiu, srai, srli, sub, sw]
matmult-int	[add, addi, bge, bne, jal, jalr, lui, lw, slli, sltiu, sw]
md5sum	[add, addi, and, andi, beq, bge, bgeu, blt, bltu, bne, jal, jalr, lui, lw, or, sb, sll, slli, sltiu, srl, srli, sub, sw, xor, xori]
minver	[add, addi, and, beq, bge, bne, jal, jalr, lui, lw, slli, slti, sltiu, sub, sw, xor]
nbody	[add, addi, and, andi, beq, bge, bne, jal, jalr, lui, lw, slli, slti, sltiu, srli, sw]
nettle-aes	[add, addi, and, andi, beq, bge, bgeu, bltu, bne, jal, jalr, lbu, lui, lw, or, sb, slli, sltiu, srli, sub, sw, xor]
nettle-sha256	[add, addi, and, andi, beq, bge, bgeu, bltu, bne, jal, jalr, lbu, lhu, lui, lw, or, sb, slli, sltiu, sltu, srli, sub, sw, xor]
nsichneu	[add, addi, beq, bge, blt, bne, jal, jalr, lui, lw, slli, sltiu, sub, sw]
picojpeg	[add, addi, and, andi, beq, bge, bgeu, blt, bltu, bne, jal, jalr, lb, lbu, lh, lhu, lui, lw, or, sb, sh, sll, slli, sltiu, sltu, sra, srai, srli, sub, sw, xor]
primecount	[add, addi, beq, bge, blt, bne, jal, jalr, lui, lw, slli, sltiu, sw]
qrduino	[add, addi, and, andi, beq, bge, bgeu, blt, bltu, bne, jal, jalr, lbu, lhu, lui, lw, or, ori, sb, sh, slli, sltiu, sltu, sra, srai, srl, srli, sub, sw, xor, xori]
sglib-combined	[add, addi, andi, beq, bge, bgeu, blt, bltu, bne, jal, jalr, lbu, lh, lui, lw, sb, sh, slli, sltiu, sltu, srai, sub, sw, xor]
slre	[add, addi, and, andi, beq, bge, bgeu, blt, bltu, bne, jal, jalr, lbu, lui, lw, or, slli, slt, sltiu, sltu, srai, sub, sw, xor]
st	[add, addi, and, bge, blt, bne, jal, jalr, lui, lw, slli, slti, sltiu, sw]
statemate	[addi, beq, bge, blt, bne, jal, jalr, lbu, lui, lw, or, sb, sh, sltiu, sub, sw]
tarfind	[add, addi, andi, beq, bge, bgeu, bltu, bne, jal, jalr, lbu, lui, lw, sb, slli, sltiu, srli, sub, sw]
ud	[add, addi, beq, bge, blt, bne, jal, jalr, lui, lw, or, slli, sltiu, sub, sw]
wikisort	[add, addi, andi, beq, bge, blt, bne, jal, jalr, lui, lw, or, slli, slt, sltiu, sltu, srai, srli, sub, sw]
armpit	[add, addi, andi, beq, bge, blt, bne, jal, jalr, lbu, lui, lw, slli, sltiu, sw]
xgboost	[addi, andi, bge, blt, jal, jalr, lui, lw, srli, sw, xor, xori]
af_detect	[add, addi, andi, beq, bge, bgeu, blt, bltu, bne, jal, jalr, lbu, lui, lw, sb, sh, slli, sltiu, srai, srli, sub, sw, xor]

and then implement the generated decision tree in C/C++ to evaluate its RISSP requirements. It can be considered as a kernel in extreme edge applications rather than a full application itself. For evaluation purposes, we use the pima indian diabetes database [39], a binary-valued data set that predicts whether a patient shows signs of diabetes. The dataset consists of 768 instances and 8 attributes.

We also select *Embench* that is an open-source benchmark suite for embedded systems. It is designed to support a wide range of embedded systems and consists of 22 C-written applications. Although it may not be perfectly aligned with the requirements of extreme edge, it offers a reasonable and structured approach to assess our methodology. The system requirements for these applications require at least 64 KB of ROM and RAM. However, an operating system and output stream are not required.

4.1 Instruction Subset Profile of Applications

Before generating a RISSP for each application, we need to characterize each application to identify the distinct instructions from the RV32E instruction set required to implement each RISSP. The next step is to explore how the number of distinct instructions is related to the codesize for different compiler optimization flags.

Figure 5 presents the profiling results of the applications from *Embench* and the three extreme edge applications. The applications

have been compiled for RV32E instruction set as baremetal (without support of `stdlib`, `libc`, `libgcc` and `startfiles`) with different compiler optimization flags (`-O0`, `-O1`, `-O2`, `-O3`, `-Oz`) using the standard `riscv32-gnu-toolchain` (version 13.2.0). Through this analysis, we identify the optimal code size with respect to the distinct instructions for each application for the generation of RISSP. The number of distinct instructions ranges from 9 to 32 instructions, and the average is 19 instructions for different optimization flags across all applications. Since the RV32E ISA has around 40 instructions, applications use only 24–86% of the full ISA. For the extreme edge applications, these numbers are 42% (*armpit*) 31% (*xgboost*), and 61% (*af_detect*). In terms of total number of static instructions across the different compiler optimization results, the average across all the applications are 2027, 1149, 1207, 1586 and 1018 for `-O0`, `-O1`, `-O2`, `-O3`, and `-Oz`, respectively. For the extreme edge applications, we observe a substantial reduction in static instructions, as we progress from `-O0` to `-Oz`. Specifically, using the `-O2` optimization level flag, we observe 75%, 74% and 69% less static instructions (reduction in codesize) than `-O0` for *armpit*, *xgboost* and *af_detect*, respectively, thus, the difference in the number of distinct instructions between `-O2` and `-Oz` is negligible.

These results indicate that there is an opportunity to improve power/area of a 32-bit RISC-V processor by eliminating unnecessary instructions and designing a customized RISSP for a target

application. Given that the `-O2` compiler flag results in minimal discrepancies in average code size with respect to distinct instructions, we select it as the basis for subsequent RISSP design for power, area and frequency analysis. **Table 3** lists the number of distinct instructions per application when compiled with `-O2` option.

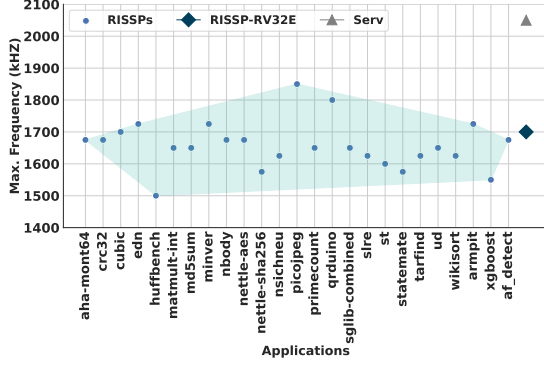


Figure 6: Maximum clock frequencies at which RISSPs, RISSP-RV32E and Serv that can operate

4.2 FlexIC Synthesis Results

The RTL of each RISSP is synthesized using *Pragmatic's* 0.6μm IGZO-based FlexIC process using a commercial EDA tool. Each RISSP is synthesized without the RF to better understand the effects of the instruction subsets in the hardware.

We also introduce two baselines to compare the RISSPs. The first baseline is an application-independent RISC-V processor supporting the full RV32E ISA (~40 instructions) generated using the RISSP methodology. This is called *RISSP-RV32E*. The second baseline is *Serv* [38], - the world's smallest 32-bit RISC-V processor. *Serv* is a bit serial processor supporting RV32I ISA, and its RF is mapped to on-chip memory rather than having a dedicated RF. *Serv* is configured to support RV32E (i.e., 16 general purpose registers rather than 32).

4.2.1 Maximum Frequency Analysis. **Figure 6** illustrates the maximum frequency (kHz) of RISSPs compared to the two baselines. To determine the maximum operating frequency, we vary the input clock frequency while adjusting design constraints in the EDA tools. Starting at 100 kHz with significant positive slack, the frequency was incremented by 25 kHz steps until reaching 3 MHz, where the design became over-constrained with negative slack. The highest frequency with positive slack is identified as the maximum. The maximum clock frequencies of RISSPs range from 1,500 to 1,850 kHz, whilst *RISSP-RV32E* and *Serv* can achieve up to 1,700 kHz and 2,050 kHz, respectively.

4.2.2 Area Analysis. **Figure 7** presents the average area of the RISSP for each application and the baselines. The average area is calculated from the NAND2-equivalent gatecounts of the processor across the range of frequencies with positive slack. RISSPs occupy 8-to-43% less area compared to *RISSP-RV32E*. The smallest RISSP (i.e., *xgboost*) is 23% larger than *Serv*.

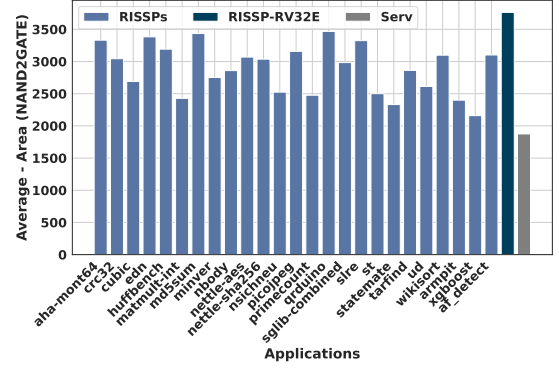


Figure 7: Average NAND2-equivalent gate count of RISSPs compared to Serv and RISSP-RV32E across the range of frequencies with positive slack

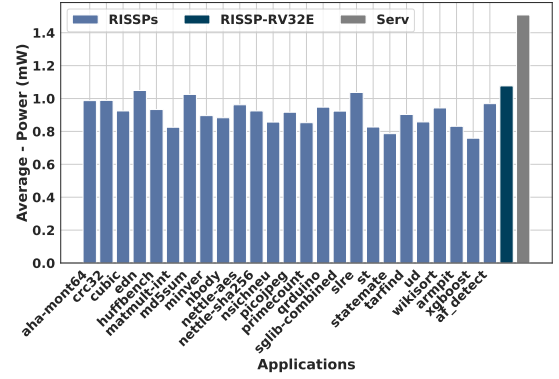


Figure 8: Average power (static + dynamic) comparison of the RISSPs, Serv and RISSP-RV32E across the range of frequencies with positive slack

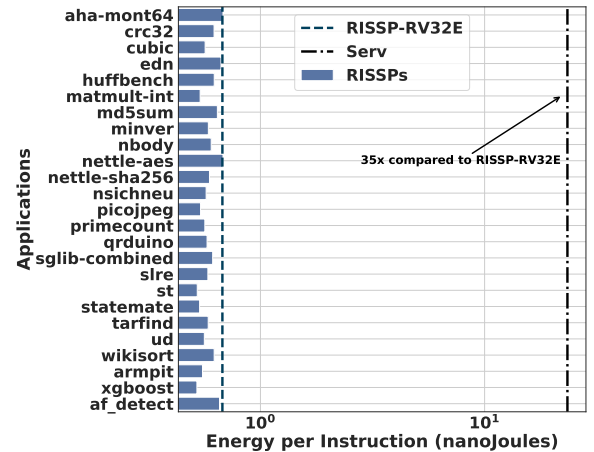


Figure 9: Energy per Instruction (EPI) in nanoJoules for RISSPs and the two baselines (RISSP-RV32E, and Serv)

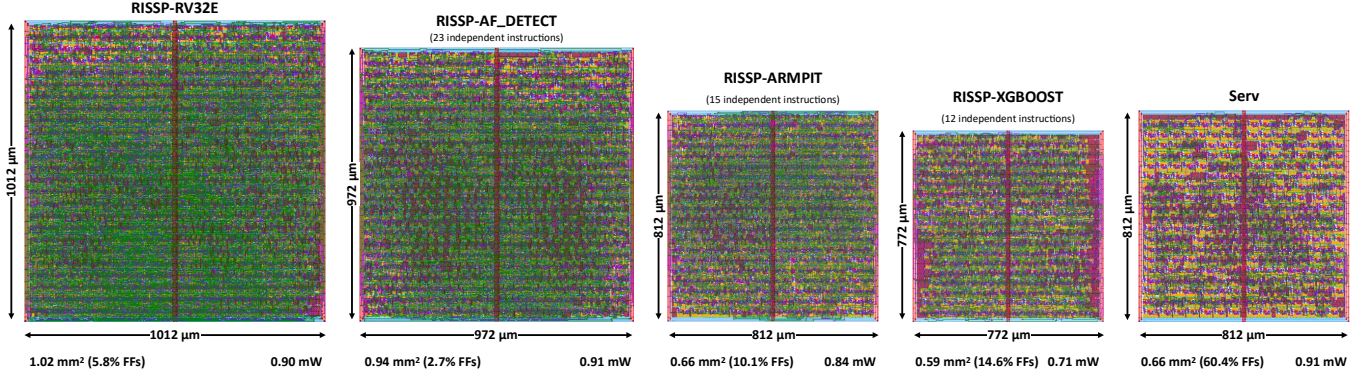


Figure 10: Layouts of the five implemented processors are shown. The leftmost layout corresponds to *RISSP-RV32E*. Next are the three RISSPs for *af_detect*, *armpit*, and *xgboost* — each representing an extreme-edge application. The rightmost layout is *Serv*. For each layout, the X and Y dimensions of the die area are shown in μm , along with the die area (in mm^2) and percentage of FFs being used in each design, on the bottom-left of each layout. The total power consumption (in mW) is indicated on the bottom-right. Additionally, for the three RISSPs, the number of distinct instructions implemented in each design is annotated.

4.2.3 Power Analysis. Figure 8 shows the average power of the RISSP for each application and the baselines. The average power is calculated in a similar manner to Section 4.2.2. RISSPs consume 3-to-30% less power than *RISSP-RV32E*. Although *Serv* has a lower gatecount compared to RISSPs and *RISSP-RV32E*, its power consumption is higher than them. For example, it consumes 40% more power than *RISSP-RV32E*. This is mainly because *Serv* has a higher proportion of flip-flops (FFs) compared to the RISSP microarchitecture, and an FF in the FlexIC process consumes 10 times more power than a NAND2 gate.

4.2.4 Energy Efficiency. Figure 9 illustrates the Energy per Instruction (EPI) for the RISSPs, *RISSP-RV32E* and *Serv*. The x-axis represents the RISSPs corresponding to each target application. Because a RISSP is a single-cycle processor, the clock cycles per instruction (CPI) for RISSPs and *RISSP-RV32E* is 1. On the other hand, *Serv* is a bit-serial processor, and has a CPI of 32 on average. The EPI is calculated by dividing the power consumption of the design point at the maximum frequency by that frequency and multiply the product by the CPI. The results highlight that *RISSP-RV32E* and RISSPs are about 35 and 40 times more energy-efficient than *Serv*. The bit-serial architecture of *Serv* leads to a high CPI resulting in much higher energy consumption per instruction.

4.3 FlexIC Physical Implementation Results

We present full physical implementation of the three extreme edge RISSPs as FlexICs and compare them to full physical implementation of the two baselines. RISSPs are generated for the instruction subsets from the applications compiled with the `-O2` flag. The physical implementation goes beyond hardware synthesis to full-fledged FlexIC layouts by floorplanning, clock tree insertion, and place and route. The goal is to show that the full physical implementation-related overheads (e.g., inserted buffers) do not hide the advantages of the RISSP philosophy (i.e., not supporting unnecessary instructions in the processor is observably beneficial).

The physical implementation is initially started at the maximum clock frequency of each processor layout at the synthesis stage.

Due to clock tree insertion and routing delays, the implementation cannot achieve the maximum clock frequency for all processors. We follow an iterative implementation process to incrementally reduce the clock frequencies until all processors became functional at 300 kHz. All processors are implemented at this frequency using typical and nominal corner conditions with a supply voltage of 3V.

Figure 10 shows the layouts of the three RISSPs, *RISSP-RV32E* and *Serv*. *RISSP-af_detect* achieves an 8% reduction in area compared to *RISSP-RV32E* whilst *RISSP-armpit* has the same area as *Serv* but reduces the area by ~35% with respect to *RISSP-RV32E*. *RISSP-xgboost* achieves even further reduction by ~11% and ~42% over *Serv* and *RISSP-RV32E*. This is because *Serv* has a high proportion of FFs (i.e., 60%), and therefore requires more clock tree buffers, which increases its area despite the fact that the synthesis results indicating that *Serv* is smaller than *RISSP-xgboost*.

The two baselines *RISSP-RV32E* and *Serv* have almost the same power consumption in spite of *Serv* being ~35% smaller than *RISSP-RV32E*. This is again because of the high proportion of FFs in *Serv* whereas FFs take up only ~6% of *RISSP-RV32E*. *RISSP-af_detect* consume almost the same power as *Serv* and *RISSP-RV32E* whilst *RISSP-armpit* and *RISSP-xgboost* consume ~8% and ~21% less power than *Serv* and *RISSP-RV32E*, respectively.

5 Generative AI based Code Retargeting

Although the concept behind the RISSP generation is to rely on the RISC-V software ecosystem rather than generating a custom software toolchain, long-lasting extreme edge applications need frequent software updates because the target applications may change. For instance, applications need to be recompiled with different compiler optimization flags, which may generate new instructions that are not supported by the RISSP under consideration.

There are two possible approaches to address this scenario. The first approach focuses on modifications at the compiler backend that needs to be modified to receive the instruction subset implemented in the RISSP and constrain the assembly code generation

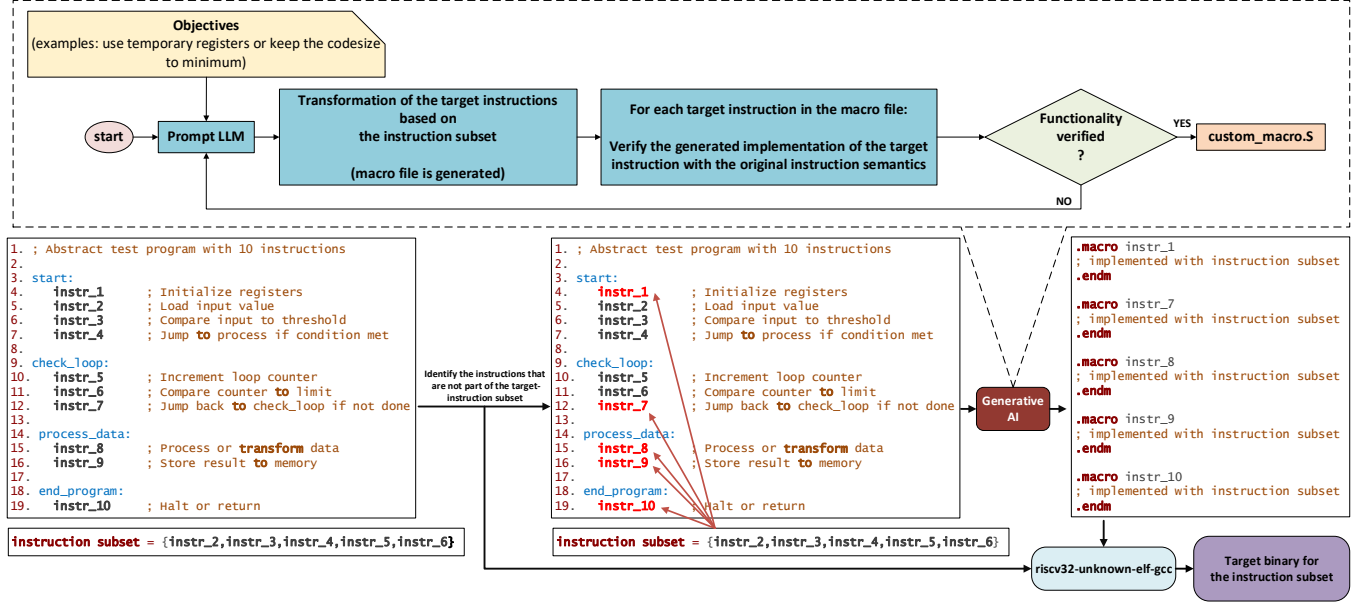


Figure 11: Generative AI based tool to implement code retargeting for RISSPs used in long-lasting extreme edge applications

to this subset only. This method is not trivial and requires significant changes in the instruction selection and code generation phases of the compiler backend. The second approach relies on the assumption that the assembly code is generated using standard compiler toolflows and a separate tool is responsible to transform the generated assembly into a version that uses the instruction subset supported by the RISSP.

We follow the second approach where we develop a tool that uses generative AI to transform the assembly code generated by the compiler targeting the full RISC-V ISA (e.g., riscv32-unknown-elf-gcc) to assembly code that uses *only* the instructions in the instruction subset. This is an important step towards reusability of RISSPs after fabrication. Figure 11 presents the retargeting framework along with a simple example. Given an application and an instruction subset, the tool is able to identify the instructions that are not included in the instruction subset. Then, with the use of ChatGPT RISC-V Assembly plugin [8], every instruction that is not part of the instruction subset is transformed into a set of instructions that are included in the instruction subset and is defined as an independent macro. The macros of the transformed instructions are stored in a macro.S file. Finally, the application is recompiled using the standard riscv32-unknown-elf-gcc with the macro.S file.

Initially, we provide the instruction subset and the instructions that need to be transformed to the ChatGPT RISC-V Assembly plugin. Objectives that restrict the solution space can be added based on the user preferences. For example, such objectives could be “allow the use of temporary registers to provide a solution”, or, “keep the codesize to minimum as you transform the instructions”. After that, the process of generating macros for the non-supported instructions from the instruction subset begins. Then, the macros are verified against the original semantics of the transformed instruction with custom test cases. If the LLM generates a macro that

cannot be functionally verified, the macro is rejected, and then another macro is requested from the LLM. This continues until a valid macro is generated. In our experiments, a valid macro can be generated on less than 10 attempts.

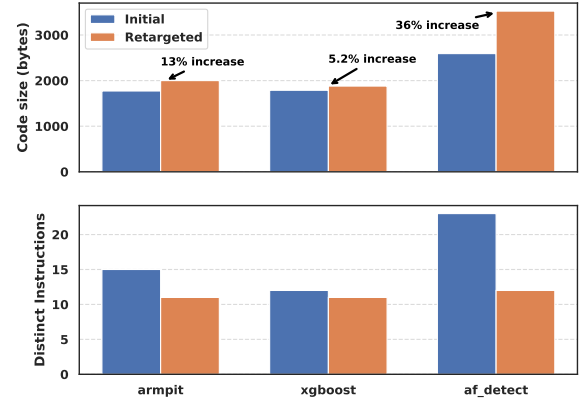


Figure 12: Comparison of code size (bytes) and the number of distinct instructions between the initial binary and the retargeted binary using LLM

To evaluate the above framework, we define a minimal instruction subset of twelve instructions {addi, add, and, xori, sll, sra, jal, jalr, blt, bltu, lw, sw} from which other instructions can be reproduced. Armpit, xgboost and af_detect can be defined as “long-lasting extreme edge applications”, as discussed in Section 4, and for this experiment they are chosen as a case study to show the potential of the framework. The framework receives as input the initial version of the target applications compiled with -O2, and attempts to transform the non-supported instructions of

the target applications to functionally relevant macro based on the instruction subset.

Figure 12 shows that the code size after the recompilation of the applications are 13%, 5.2% and 36% larger than the original version for *armpit*, *xbgboost* and *af_detect*, respectively. The number of distinct instructions shows minimal variation for *armpit* and *xbgboost*, while the number of distinct instructions for *af_detect* is reduced from 23 to 12. This experiment demonstrates the feasibility of this approach to efficiently retarget code but with an overhead of larger code size and potentially slower runtime of the application.

6 Conclusions

We have proposed a methodology for automatically generating a single-cycle 32-bit RISC-V instruction subset processor (RISSP) to be deployed as a FlexIC to meet the computing requirements of the extreme edge. The methodology constructs a RISSP supporting only the instruction subset identified during the characterization of an extreme edge application or domain. The semantics of each instruction in the RISC-V ISA are translated into a formally verified and discrete, hardware block and stored in a library. The hardware blocks of the instruction subset are extracted from the library, stitched together to build the core of the RISSP, which is then connected to the rest of the pre-verified processor components to generate the RISSP. The RISSP methodology reduces the time-to-market of custom processors by integrating verification as a primary design principle. The synthesis results have shown that RISSPs can achieve 8-to-43% reduction in area and 3-to-30% reduction in power compared to a processor supporting the full RISC-V ISA, and are on average about 40 times more energy efficient than *Serv*- the world's smallest 32-bit RISC-V processor. When implemented as FlexICs, the RISSPs targeting extreme edge applications can achieve up to 42% and 21% less area and power.

The proposed methodology can support custom instructions that can be added to the pre-verified ISA hardware library like other instructions in the ISA. Also, the methodology can be extended to generate pipelined RISSPs if higher clock frequencies are required but is not suitable for generating complex processors that have deeper superscalar and out-of-order execution pipelines. Extreme edge applications do not require such high performance processors, so our focus has been to develop a microcontroller-class RISSP that is a single-cycle scalar in-order execution machine providing adequate performance for applications at extreme edge.

Acknowledgments

This work is partially funded by GRASP EU Project No 101134936. The authors would like to thank John Biggs, a co-founder of Arm and a Pragmatic fellow, for his valuable inputs on physical implementation analysis, and also Bruce Hoult from the RISC-V sub-reddit forum for his comments on macro retargeting.

References

- [1] Ashiq Ahamed, Peng Huang, Joshua Young, Alejandro Gallego-Schmid, Richard Price, and Michael P. Shaver. 2024. Technical and environmental assessment of end-of-life scenarios for plastic packaging with electronic tags. *Resources, Conservation and Recycling* 201 (2024), 107341.
- [2] Kubilay Atasu, Laura Pozzi, and Paolo Ienne. 2003. Automatic application-specific instruction-set extensions under microarchitectural constraints. In *Proceedings of the 40th annual Design Automation Conference*. 256–261.
- [3] John Biggs, James Myers, Jędrzej Kufel, Emre Ozer, Simon Craske, Antony Sou, Catherine Ramsdale, Ken Williamson, Richard Price, and Scott White. 2021. A natively flexible 32-bit Arm microprocessor. *Nature* 595, 7868 (2021), 532–536.
- [4] Nathaniel Bleier, Calvin Lee, Francisco Rodriguez, Antony Sou, Scott White, and Rakesh Kumar. 2022. Flexicores: Low footprint high yield field reprogrammable flexible microprocessors. In *Annual International Symposium on Computer Architecture (ISCA)*. 831–846.
- [5] Nathaniel Bleier, M Husnain Mubarik, Suman Balaji, Francisco Rodriguez, Antony Sou, Scott White, and Rakesh Kumar. 2023. Exploiting short application lifetimes for low cost hardware encryption in flexible electronics. In *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.
- [6] Nathan Bleier, John Sartori, and Rakesh Kumar. 2021. Property-driven automatic generation of reduced-isa hardware. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 349–354.
- [7] Hikmet Çeliker, Antony Sou, Brian Cobb, Wim Dehaene, and Kris Myny. 2022. Flex6502: a flexible 8b microprocessor in 0.8 μm metal-oxide thin-film transistor technology implemented with a complete digital design flow running complex assembly code. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 65. IEEE, 272–274.
- [8] ChatGPT. n.d.. ChatGPT RISC-V Assembly. [Online]. Available: <https://chatgpt.com/g/g-sKaT8XCS-risc-v-assembly>.
- [9] Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 785–794.
- [10] Hari Cherupalli, Henry Duwe, Weidong Ye, Rakesh Kumar, and John Sartori. 2017. Bespoke processors for applications with ultra-low area and power constraints. In *Proceedings of the 44th Annual International Symposium on Computer Architecture*. 41–54.
- [11] Júlio Costa, Vincent Barlier, Hellen Norman, Emre Ozer, Feras Alkhalil, and Richard Price. 2023. 11-5: Invited Paper: Evolving into an Era of Natively Flexible Smart Systems. In *SID Symposium Digest of Technical Papers*, Vol. 54. Wiley Online Library, 136–139.
- [12] eeNews. 2025. How much 'Moore' do we need? New technologies for the extreme edge! [Online]. Available: <https://www.eenewseurope.com/en/how-much-moore-do-we-need-new-technologies-for-the-extreme-edge/>.
- [13] GitHub. . RISC-V Formal Verification Framework. <https://github.com/SymbioticEDA/riscv-formal>.
- [14] GitHub. 2017. SymbiYosys (SBY). <https://github.com/YosysHQ/SymbiYosys>.
- [15] GitHub. 2018. RISC-V Architecture Test SIG. <https://github.com/riscv-non-isa/riscv-arch-test>.
- [16] GitHub. 2019. RISCOF: RISC-V Architectural Compatibility Framework. <https://github.com/riscv/riscof>.
- [17] GitHub. 2020. MCY: Model Checking for Yosys. <https://github.com/YosysHQ/mcy>.
- [18] Ricardo E Gonzalez. 2000. Xtensa: A configurable and extensible processor. *IEEE micro* 20, 2 (2000), 60–70.
- [19] Fortune Business Insights. n.d.. Wound Care Devices Market Size. [Online]. Available: <https://www.fortunebusinessinsights.com/industry-reports/wound-care-devices-market-100388>.
- [20] Konstantinos Iordanou, Timothy Atkinson, Emre Ozer, Jędrzej Kufel, Grace Ali-gada, John Biggs, Gavin Brown, and Mikael Luján. 2024. Low-cost and efficient prediction hardware for tabular data using tiny classifier circuits. *Nature Electronics* 7, 5 (2024), 405–413.
- [21] Markets and Markets. n.d.. Environmental Monitoring Market Size. [Online]. Available: <https://www.marketsandmarkets.com/Market-Reports/environmental-monitoring-market-216846315.html>.
- [22] Heinrich Meyr. 2004. Application specific instruction-set processors (ASIP's) for wireless communications: design, cost, and energy efficiency vs. flexibility. In *2004 International Symposium on System-on-Chip, 2004. Proceedings.* IEEE, 1–2.
- [23] Kris Myny, Yi-Cheng Lai, Nikolaos Papadopoulos, Florian De Roose, Marc Ameys, Myriam Willegems, Steve Smout, Soeren Steudel, Wim Dehaene, and Jan Genoe. 2017. 15.2 A flexible ISO14443-A compliant 7.5 mW 128b metal-oxide NFC barcode tag with direct clock division circuit from 13.56 MHz carrier. In *2017 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 258–259.
- [24] Kris Myny, Steve Smout, Maarten Rockel, Ajay Bhoolokam, Tung Huei Ke, Soeren Steudel, Koji Obata, Marko Marinkovic, Duy-Vu Pham, Arne Hoppe, et al. 2014. 30.1 8b thin-film microprocessor using a hybrid oxide-organic complementary technology with inkjet-printed P 2 ROM memory. In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*. IEEE, 486–487.
- [25] Kris Myny, Erik van Veenendaal, Gerwin H. Gelinck, Jan Genoe, Wim Dehaene, and Paul Heremans. 2012. An 8-Bit, 40-Instructions-Per-Second Organic Microprocessor on Plastic Foil. *IEEE Journal of Solid-State Circuits* 47, 1 (2012), 284–291. <https://doi.org/10.1109/JSSC.2011.2170635>
- [26] Arokia Nathan, Arman Ahnood, Matthew T. Cole, Sungsik Lee, Yuji Suzuki, Pritesh Hirralal, Francesco Bonaccorso, Tawfique Hasan, Luis Garcia-Gancedo, Andriy Dyadyusha, Samiul Haque, Piers Andrew, Stephan Hofmann, James Moultrie, Daping Chu, Andrew J. Flewitt, Andrea C. Ferrari, Michael J. Kelly, John Robertson, Gehan A. J. Amarutunga, and William I. Milne. 2012. Flexible

- Electronics: The Next Ubiquitous Platform. *Proc. IEEE* 100, Special Centennial Issue (2012), 1486–1517. <https://doi.org/10.1109/JPROC.2012.2190168>
- [27] Emre Ozer, Jędrzej Kufel, John Biggs, Gavin Brown, James Myers, Anjit Rana, Antony Sou, and Catherine Ramsdale. 2019. Bespoke machine learning processor development framework on flexible substrates. In *2019 IEEE international conference on flexible and printable sensors and systems (FLEPS)*. IEEE, 1–3.
- [28] Emre Ozer, Jędrzej Kufel, John Biggs, James Myers, Charles Reynolds, Gavin Brown, Anjit Rana, Antony Sou, Catherine Ramsdale, and Scott White. 2020. Binary neural network as a flexible integrated circuit for odour classification. In *2020 IEEE International Conference on Flexible and Printable Sensors and Systems (FLEPS)*. IEEE, 1–4.
- [29] Emre Ozer, Jędrzej Kufel, John Biggs, Anjit Rana, Francisco J Rodriguez, Thomas Lee-Clark, Antony Sou, Catherine Ramsdale, Scott White, Suresh K Garlapati, Garlapati, Valliappan Palaniappan, Aiman Rahmanudin, Venuskishnan Koman-duri, Glenn S. Saez, Sankara Gollu, Gavin Brown, Piotr Dudek, Krishna C. Persaud, Michael L Turner, Stephanie Murray, Susan Bates, Robert Treloar, Brian Newby, and Jane Ford. 2023. Malodour classification with low-cost flexible electronics. *Nature Communications* 14, 1 (2023), 777.
- [30] Emre Ozer, Jędrzej Kufel, James Myers, John Biggs, Gavin Brown, Anjit Rana, Antony Sou, Catherine Ramsdale, and Scott White. 2020. A hardwired machine learning processing engine fabricated with submicron metal-oxide thin-film transistors on a flexible substrate. *Nature Electronics* 3, 7 (2020), 419–425.
- [31] Emre Ozer, Jędrzej Kufel, Shvetank Prakash, Alireza Raisiardi, Olof Kindgren, Ronald Wong, Nelson Ng, Damien Jausseran, Feras Alkhalil, David Kong, et al. 2024. Bendable non-silicon RISC-V microprocessor. *Nature* 634, 8033 (2024), 341–346.
- [32] Emre Ozer, Jędrzej Kufel, Elena Punskeya, Igor Drovkov, Ari Ercole, Hiral Radia, Thom Smith, John Biggs, Francisco Rodriguez, Grace Aligada, and Adam Rearden. 2024. A Custom-designed Atrial Fibrillation Detection hardware on a Flexible Substrate. In *2024 IEEE International Conference on Flexible and Printable Sensors and Systems (FLEPS)*. IEEE, 1–4.
- [33] Velu Pillai, Igor DS Miranda, Tejas Musale, Mugdha Jadhao, Paulo CR Souza Neto, Zachary Susskind, Alan TL Bacellar, Mael Lhostis, Priscila MV Lima, Diego LC Dutra, Eugene B John, Mauricio Breternitz, Felipe M. G. França, Emre Ozer, and Lizy K John. 2024. arrWNN: Arrhythmia-Detecting Weightless Neural Network FlexIC. In *2024 IEEE International Flexible Electronics Technology Conference (IFETC)*. IEEE, 1–4.
- [34] Pragmatic. 2025. FlexIC Platform Gen3. <https://www.pragmaticsemi.com/foundry/flexic-platform-gen-3>. [Online]. Available: <https://www.pragmaticsemi.com/foundry/flexic-platform-gen-3>.
- [35] Allied Market Research. n.d. FMCG Market Expected to Reach \$18,939.4 Billion by 2031. [Online]. Available: <https://www.alliedmarketresearch.com/press-release/fmcg-market.html>.
- [36] Transparency Market Research. n.d. Smart Patches Market. [Online]. Available: <https://www.transparencymarketresearch.com/smart-patches-market.html>.
- [37] riscv-software src. n.d. Spike ISA Sim - GitHub - riscv-software-src/riscv-isa-sim. [Online]. Available: <https://github.com/riscv-software-src/riscv-isa-sim>.
- [38] Serv. 2019. Bit-serial RISC-V CPU - GitHub - olofk/serv. [Online]. Available: <https://github.com/olofk/serv>.
- [39] Jack W Smith, James E Everhart, William C Dickson, William C Knowler, and Robert Scott Johannes. 1988. Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. In *Proceedings of the annual symposium on computer application in medical care*. 261.
- [40] Johan Van Praet, Gert Goossens, Dirk Lanneer, and Hugo De Man. 1994. Instruction set definition and instruction selection for ASIPs. In *Proceedings of 7th International Symposium on High-Level Synthesis*. IEEE, 11–16.
- [41] Andrew Waterman, Yunsup Lee, David A Patterson, and Krste Asanovic. 2011. The risc-v instruction set manual, volume i: Base user-level isa. *EECS Department, UC Berkeley, Tech. Rep. UCB/EECS-2011-62* 116 (2011), 1–32.
- [42] Piotr Zbigniew Wiecek, Krzysztof Starecki, Krzysztof Golofit, Maciej Radtke, and Marcin Pilarz. 2023. A thin elastic NFC Forum type 1 compatible RFID tag. *IEEE Journal of Solid-State Circuits* 59, 3 (2023), 935–946.
- [43] Mohammad Zulqarnain, Stefano Stanzione, Ganesh Rathinavel, Steve Smout, Myriam Willegems, Kris Myny, and Eugenio Cantatore. 2020. A flexible ECG patch compatible with NFC RF communication. *npj Flexible Electronics* 4, 1 (2020), 13.